Chapter 3 Function

Introduction and Defining Functions

- A complex problem is often easier to solve by dividing it into several smaller parts, each of which can be solved by itself.

- These parts are made into ***functions*** in C++. **main()** then uses these functions to solve the original problem.

- Experience has shown that the best way to develop and maintain large programs is to construct it from smaller pieces(Modules). This technique Called "**Divide and Conquer**".

Advantages of Functions

- Functions separate the concept (what is done) from the implementation (how it is done).
- Functions make programs easier to understand.
- Functions can be called several times in the same program, allowing the code to be reused
- Easer To:
  - Design, Build, Debug, Extend and Modify

Introduction and Defining Functions

- C++ allows the use of both internal (user-defined) and external functions. External functions (e.g., **abs**, **ceil**, **rand**, **sqrt**, etc.) are usually grouped into specialized libraries (e.g., **iostream**, **stdlib**, **math**, etc.)
- User-Defined Functions in C++ programs usually have the following form:
  - // include statements
  - // function prototypes
  - // main() function
  - // function definitions
- The function prototype declares the input and output parameters of the function.
- The **function prototype** has the following syntax:

  <type> <function name>(<type list>);
- Example: A function that returns the absolute value of an integer is:

  int absolute(int);

Introduction and Defining Functions

- A **function definition** in C++ has the following syntax:

```
<type> <function name>(<parameter list>){
    <local declarations>
    <sequence of statements>
}
```

- For example: Definition of a function that computes the absolute value of an integer:

```
int absolute(int x){
    if (x >= 0)   return x;
    else
    return -x;
}
```

- If a function definition is placed in front of main(), there is no need to include its function prototype.
- A **function call** has the following syntax:

```
<function name>(<argument list>)
```

  - Example: int distance = absolute(-5);

C++ Variables Scope in Functions

- The scope of the variables can be broadly be classified as Local and Global

Local Variables:

- The variables defined local to the **block** of the function would be accessible only within the block of the function and not outside the function. Such variables are called local variables. That is in other words the scope of the local variables is limited to the function in which these variables are declared.
- Let us see this with a small example:

```
#include <iostream.h>
int exforsys(int,int);//definition is given in the next slide
void main( ){
        int b,s=5,u=6;
        b=exforsys(s,u);
        cout << "n The Output is:" << b;
}}
```

C++ Variables Scope in Functions


```
int exforsys(int x,int y)
{
        int z;
        z=x+y;
        return(z);
}
```

- In the above program the variables x, y, z are accessible only inside the function exforsys( ) and their scope is limited only to the function exforsys( ) and not outside the function. Thus the variables x, y, z are local to the function exforsys. Similarly one would not be able to access variable b inside the function exforsys as such. This is because variable b is local to function main.

Global Variables:

- Global variables are one which are visible in any part of the program code and can be used within all functions and outside all functions used in the program. The method of declaring global variables is to declare the variable outside the function or block.

C++ Variables Scope in Functions

Global variable

```
#include <iostream.h>
    int x,y,z;        //Global Variables
    float a,b,c;      //Global Variables
   void main( )
   {
      int s,u;      //Local Variables
      float w,q;        //Local Variables
      ...
      ...
   }
```

- In the above the integer variables x, y and z and the float variables a, b and c which are declared outside the block are global variables and the integer variables s and u and the float variables w and q which are declared inside the function block are called local variables.
- Thus the scope of global variables is between the **point of declaration** and the **end of compilation** unit whereas scope of local variables is between the **point of declaration** and the end of **innermost enclosing** compound statement.

Passing Variables

Arguments/Parameters

- There is one-to-one correspondence between the arguments in a function call and the parameters in the function definition.
  - *int argument1; double argument2;*
  - *// function call (in another function, such as main)*
  - *result = thefunctionname(argument1, argument2);*
- *// function definition*
  - *int thefunctionname(int parameter1, double parameter2){*
  - *// Now the function can use the two parameters*
  - *// parameter1 = argument 1, parameter2 = argument2*

Parameter Passing - pass by value

- the default in C++ (except when passing arrays as arguments to functions)
- formal parameter receives the value of the argument
- changes to the formal parameter do not affect the argument
- a copy of the value is passed from the caller-function to the called-function
- Any change to the copy does not affect the original value in the caller function
- Advantages, prevents side effect, resulting in reliable software

Example


```
#include <iostream>
using namespace std;
int fact(int);    //function prototype
int main()
{   int n, factorial;
    cin >> n;
    if(n>=0)
    {   factorial = fact(n);//function call
    cout << n <<"! is " << factorial << endl;
    }//end if
return 0;
}//end main
```

```
int fact(int n)  //function header
{       int nfact = 1;
    while(n>1)
    {       nfact = nfact*n;
            n--;
    }//end while block
    return(nfact);
} //end fact
```

Parameter Passing - pass by reference

- append an **&** to the parameter data type in both the function prototype and function header
- formal parameter receives the ***address*** of the argument
- any changes to the formal parameter directly change the value of the argument

- We introduce reference-parameter, to perform call by reference. The caller gives the called function the ability to directly access the caller's value, and to modify it.
- A reference parameter is an alias for it's corresponding argument, it is stated in c++ by "flow the parameter's type" in the function prototype by an ampersand(&) also in the function definition-header.

- Advantage: performance issue

Format


```
void  function_name (type &);// prototype


      main()
      {
            -----

            ------
      }
void function_name(type &parameter_name)
```

Example


```
// passing parameters by reference
#include <iostream>
using namespace std;

void duplicate (int& a, int& b,
int& c)
{
  a*=2;
  b*=2;
  c*=2;
}

int main ()
{
  int x=1, y=3, z=7;
  duplicate (x, y, z);
  cout << "x=" << x << ", y=" << y
<< ", z=" << z;
  return 0;
}
```

```
x=2, y=6, z=14
```

```
x=2, y=6, z=14
```

Stack

- The memory that a program uses is typically divided into four different areas:
  - **The code area:** where the compiled program sits in memory.
  - **The global area**: where global variables are stored.
  - **The heap:** where dynamically allocated variables are allocated from.
  - **The stack**: where parameters and local variables are allocated from.

The stack

- Consider a stack of plates in a cafeteria. Because each plate is heavy and they are stacked, you can really only do one of three things:
  1) Look at the surface of the top plate
  2) Take the top plate off the stack
  3) Put a new plate on top of the stack

Cont..

- In computer programming, a stack is a container that holds other variables (much like an array). However, whereas an array lets you access and modify elements in any order you wish, a stack is more limited. The operations that can be performed on a stack are identical to the ones above:

1) Look at the top item on the stack (usually done via a function called top())
2) Take the top item off of the stack (done via a function called pop())
3) Put a new item on top of the stack (done via a function called push())

Inline functions

- Each time you call a function in a C++ program, the computer must do the following:
  - Remember where to return when the function eventually ends
  - Provide memory for the function's variables
  - Provide memory for any value returned by the function
  - Pass control to the function
  - Pass control back to the calling program
- This extra activity constitutes the overhead, or cost of doing business, involved in calling a function
- An inline function is a small function with no calling overhead
- Overhead is avoided because program control never transfers to the function

Inline functions

- A copy of the function statements is placed directly into the compiled calling program
- The inline function appears prior to the main(), which calls it
- Any inline function must precede any function that calls it, which eliminates the need for prototyping in the calling function
- When you compile a program, the code for the inline function is placed directly within the main() function
- You should use an inline function only in the following situations:
  - When you want to group statements together so that you can use a function name
  - When the number of statements is small (one or two lines in the body of the function)
  - When the function is called on few occasions

Inline functions

- An interesting mechanism to increase the performances of a program consists in replacing function calls by the function itself. To specify to the compiler to do that, we can use the inline keyword :

```
#include <iostream>
inline double dumb1(double x) { return 17*x; }
double dumb2(double x) { return 17*x; }
int main(int argc, char **argv) {
    double x = 4;
    cout << x << '\n';
    x = dumb1(x);
    cout << x << '\n';
    x = dumb2(x);
    cout << x << '\n';
}
```

Overloaded functions.

- In C++ two different functions can have the same name if their parameter types or number are different. Example:

```
// overloaded function
#include <iostream>
using namespace std;

int operate (int a, int b)
{
  return (a*b);
}

float operate (float a, float b)
{
  return (a/b);
}

int main ()
{
  int x=5,y=2;
  float n=5.0,m=2.0;
  cout << operate (x,y);
  cout << "\n";
  cout << operate (n,m);
  cout << "\n";
  return 0;
}
```

```
10
2.5
```

Cont..

- In this case we have defined two functions with the same name, operate, but one of them accepts two parameters of type int and the other one accepts them of type float. The compiler knows which one to call in each case by examining the types passed as arguments when the function is called. If it is called with two ints as its arguments it calls to the function that has two int parameters in its prototype and if it is called with two floats it will call to the one which has two float parameters in its prototype.
- In the first call to operate the two arguments passed are of type int, therefore, the function with the first prototype is called; This function returns the result of multiplying both parameters. While the second call passes two arguments of type float, so the function with the second prototype is called. This one has a different behavior: it divides one parameter by the other. So the behavior of a call to operate depends on the type of the arguments passed because the function has been overloaded.
- Notice that a function cannot be overloaded only by its return type. At least one of its parameters must have a different type.

Cont..

- C++ enables several functions of the same name to be defined, as long as these functions have different sets of parameters
  - Different parameter types or
  - Different number of parameters or
  - Different order of the parameter types
- This capability is called **function overloading**. When an overloaded function is called, the C++ compiler selects the proper function by examining the number, types and order of the arguments in the call.
- Function overloading is commonly used to create several functions of the same name that perform similar tasks, but on different data types. For example, many functions in the math library are overloaded for different numeric data types

Recursive function call

- A function can call itself in its definition statement. We call such a scheme a recursive function. In fact this is possible because at each call, new variables are allocated in the memory.
- Example:

```
#include <iostream>
int fact(int k) {
    cout << k << '\n';
    if(k == 0) return 1;
       else return k*fact(k-1);
}
void main() {
    int n = fact(4);
    cout << '\n' << n << "\n";
}
```

Recursive function call

- Consider this function.

```
void myFunction( int counter){
if(counter == 0)
     return;
else  {
       cout <<counter<<endl;
       myFunction(--counter);
   return;
   }
}
```

- Every recursion should have the following characteristics.
  - A simple base case which we have a solution for and a return value. Sometimes there are more than one base cases.
  - A way of getting our problem closer to the base case. I.e. a way to chop out part of the problem to get a somewhat simpler problem.
  - A recursive call which passes the simpler problem back into the function.

Recursive function call

- The key to thinking recursively is to see the solution to the problem as a **smaller version** of the same problem.

- The key to solving recursive programming requirements is to imagine that your function does what it does even before you have actually finish writing it. You must pretend the function does its job and then use it to solve the more complex cases. Here is how.
  - Identify the base case(s) and what the base case(s) do.
  - Return the correct value for the base case.
  - Your recursive function will then be comprised of an if-else statement where the base case returns one value and the non-base case(s) recursively call(s) the same function with a **smaller** parameter or set of data.

Recursive function call

- Let's consider writing a function to find the factorial of an integer. For example 7! equals 7*6*5*4*3*2*1 .
- But we are also correct if we say 7! equals 7*6!.
- In seeing the factorial of 7 in this second way we have gained a valuable insight. We now can see our problem in terms of a simpler version of our problem and we even know how to make our problem progressively more simple.
- We have also defined our problem in terms of itself. I.e. we defined 7! in terms of 6!. This is the essence of recursive problem solving. Now all we have left to do is decide what the base case is. What is the simplest factorial? 1!. 1! equals 1.
- Let's write the factorial function recursively.
- Int myFactorial(int integer)

```
{
if( integer == 1)
    return 1;
else
      {
      return (integer * (myFactorial(integer-1)));
      }
}
```

Recursive function call


```
Fibonacci(int n) {
    if (n==0)
        return 0;
    if (n==1)
        return 1;
    return( Fibonacci(n-2) + Fibonacci(n-1) );
}
```

A recursive version of function to determine if an input is prime.

```
bool isPrime(int p, int i=2) {
    if (i==p)
        return 1;//or better
     if (i*i>p)
        return 1;
    if (p%i == 0)
        return 0;
    return isPrime (p, i+1);
 }
```

Example

- Example to print numbers counting down:

```
void print(int p) {
    if (p==0) return;
      cout<<p; print(p-1);
    return;
}
```

- Example to print counting up

```
void print(int p) {
     if (p==0) return;
       print(p-1);
    cout<<p;
    return;
}
```

Factorial

```
long fact( long n ){
        long ans = 1;
        for(long i = n; i > 1; i--)
                ans *= i;
        return ans;
 }
```